

UseCaseAgent

Ein modularer KI-Agent zur nutzerzentrierten Beschreibung von Use Cases

Projekt	UseCaseAgent — professionelle Web-App im ChatGPT-/Claude-Code-Stil
Stack	FastAPI (Backend, SSE-Streaming) · Vanilla-JS-Frontend (kein Build) · OpenRouter
Modelle	Claude Opus 4.8 · GPT-5.6 Sol · GPT-5.6 Terra — je vier Denkstufen (Niedrig · Mittel · Hoch · Max)
Start	py run.py → http://localhost:8000
Dokument	Projektdokumentation mit Rundgang und Screenshots

Was ist UseCaseAgent?

Use Cases sauber zu formulieren ist Handwerk: Man muss die richtigen Akteure erkennen, Ziele von Abläufen trennen, Vor- und Nachbedingungen sowie Ausnahmefälle bedenken und das Ganze testbar und eindeutig aufschreiben. **UseCaseAgent** nimmt einer nicht-technischen Person genau diese Arbeit ab: Sie beschreibt ihre Idee in Alltagssprache, der Agent führt ein kurzes Interview und liefert am Ende eine standardisierte Use-Case-Beschreibung im Cockburn-Stil.

Statt eines einzigen „Black-Box“-Prompts arbeitet im Hintergrund ein **Team aus fünf spezialisierten Agenten**, die sich die Arbeit sichtbar zuspiesen. Jeder Denkschritt, jede Übergabe und jede erzeugte bzw. bearbeitete Datei wird live im Chat dargestellt — nachvollziehbar wie bei einem Entwickler-Werkzeug, aber bedienbar wie ein moderner Chat-Assistent.

Kernfunktionen

- **Echter Chat-Agent mit sichtbarem Denkprozess:** jeder modulare Schritt (Interviewer → Akteur-Analyst → Use-Case-Architekt → Qualitätsmanager → Use-Case-Optimierer) ist eine ein-/ausklappbare Zeile; Übergaben werden als eigene Ereignisse dargestellt.
- **Dateien im Claude-Code-Stil:** „Erstellt UC-001...json +105 ›“ mit Zeilenzähler, Diff-Ansicht (grün/rot) und aufklappbarer Inhalts-Vorschau.
- **Modell- & Denkstufen-Picker:** Auswahl von Modell und Denkstufe (Niedrig · Mittel · Hoch · Max) per Hover-Flyout.
- **Live-Qualitätsbewertung:** der Qualitätsmanager vergibt einen Score (z. B. 84/100); der Optimierer behebt die offenen Punkte, danach folgt eine Endprüfung — bei Bedarf in mehreren Runden.
- **Spracheingabe** (Web Speech API, Chrome/Edge), frei wählbarer **Ausgabeordner** (sonst Download) und ein **Beispiel-Prompt-Generator**.
- **Vorschau-Panel:** der fertige Use Case wird als gerendertes Markdown direkt neben dem Chat angezeigt.
- **Chat-Historie**, ein-/ausklappbare Navigation und **Hell/Dunkel-Theme** (Akzentfarbe #009A9B).
- **Datenschutz:** der OpenRouter-Key liegt ausschließlich lokal im Browser und wird pro Anfrage an das lokale Backend gesendet.

Der modulare Agenten-Ablauf

Nutzer-Idee

- Interviewer versteht die Idee, stellt gezielte Rückfragen
- (Reife-Prüfung) genug Infos für einen Use Case?
- Akteur-Analyst leitet primäre / sekundäre / System-Akteure + Ziele ab
- Use-Case-Architekt baut die standardisierte Beschreibung (Cockburn-Stil)
- Qualitätsmanager prüft Vollständigkeit, Eindeutigkeit, Testbarkeit → Score
- Use-Case-Optimierer behebt die offenen Punkte
- (Endprüfung) erneute Kontrolle; ggf. weitere Optimierungsrunde
- Dateien finale UC-XXX.md + UC-XXX.json; TMP-Entwürfe gelöscht

Jeder Agent streamt seine **Gedanken** (Reasoning) und seine sichtbare Antwort. Zwischenstände werden als temporäre `.tmp.json`-Entwürfe gespeichert und am Ende automatisch aufgeräumt — übrig bleiben nur die geprüften Enddateien.

Technologie-Stack

Schicht	Technologie
Backend	Python · FastAPI · Uvicorn · Pydantic · httpx
Streaming	Server-Sent Events (SSE) — Denken, Übergaben und Ergebnisse live
Frontend	Vanilla HTML/CSS/JS (bewusst ohne Build-Tooling / Node)
Modell-Provider	OpenRouter (OpenAI-kompatibel)
Spracheingabe	Web Speech API (Chrome/Edge)
Persistenz	Chats als JSON; Ausgaben in frei wählbarem Ordner oder Download

Rundgang durch die Anwendung

1 · Startbildschirm

Aufgeräumter Einstieg im ChatGPT-/Claude-Code-Stil: links die Chat-Historie, zentral die Aufforderung „Beschreibe eine Idee — ich baue den Use Case.“ und die Prompt-Bar mit Ausgabeordner-Chip, Modell-Picker, Mikrofon und Senden-Knopf. Der Button „Beispiel-Prompt generieren“ erzeugt auf Wunsch eine laienhafte Feature-Idee.

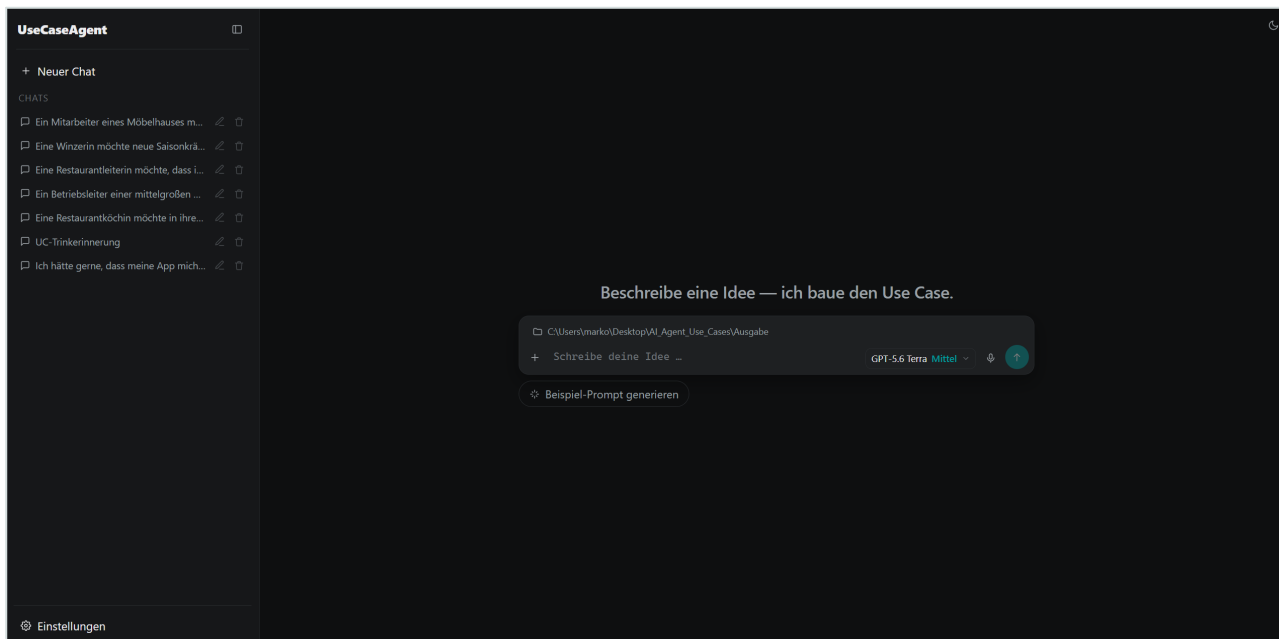


Abb. 1 — Startbildschirm im dunklen Theme.

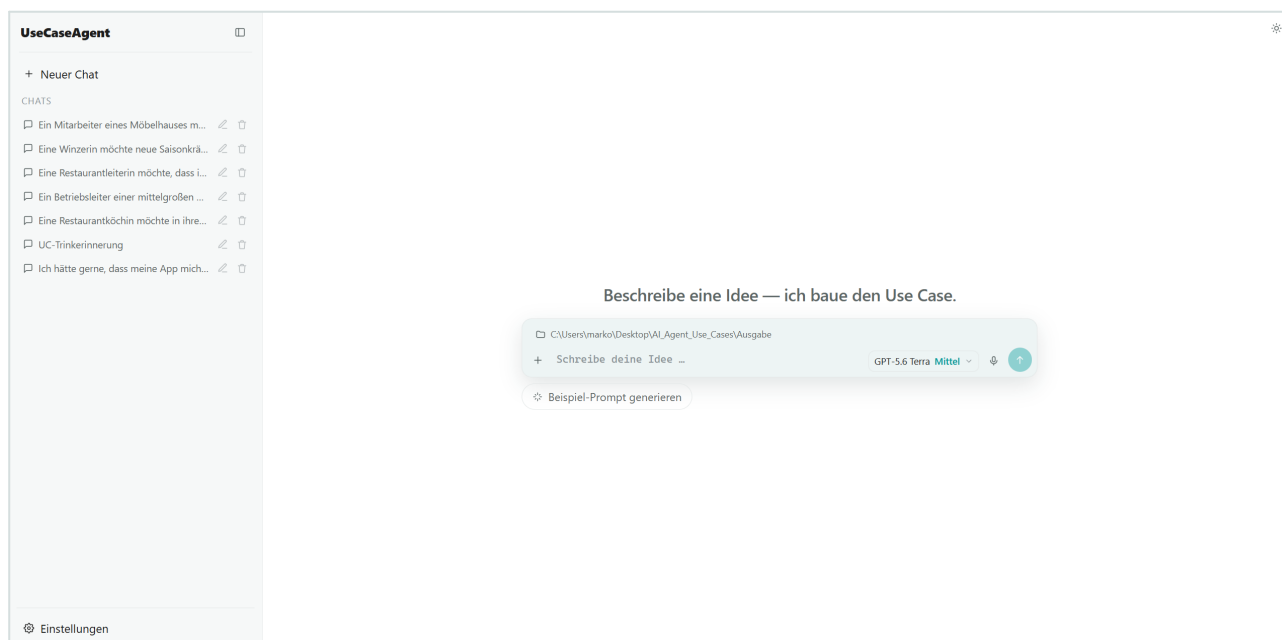


Abb. 2 — Dieselbe Oberfläche im hellen Theme.

2 · Seitenleiste & Navigation

Die ein-/ausklappbare Seitenleiste bündelt „Neuer Chat“, die Liste der bisherigen Use-Case-Chats (jeweils umbenennbar/löschbar) und den Zugang zu den Einstellungen.

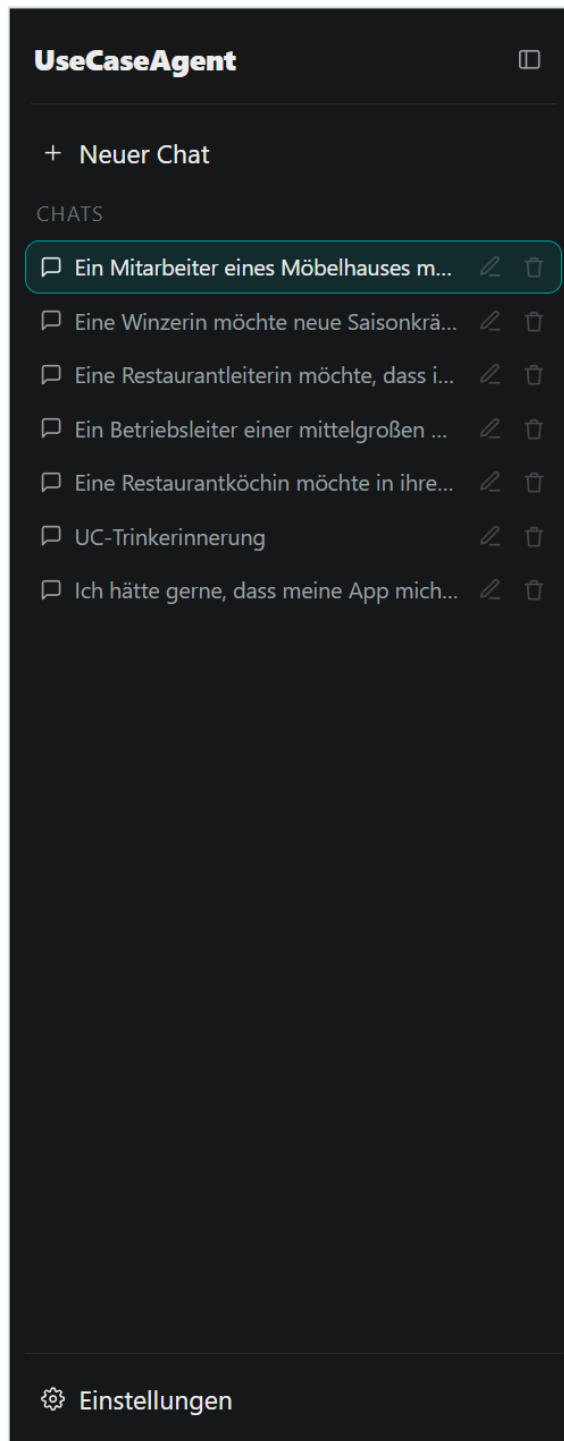


Abb. 3 — Seitenleiste mit Chat-Historie und Einstellungen.

3 · Einstellungen: API-Key & Agenten-Vorgaben

Im Einstellungs-Dialog wird der **OpenRouter-API-Key** hinterlegt (nur lokal im Browser). Darunter lassen sich für **jeden Agenten dauerhafte Zusatz-Vorgaben** definieren — z. B. eigene Regeln für Interviewer, Akteur-Analyst, Use-Case-Architekt, Qualitätsmanager oder Use-Case-Optimierer.

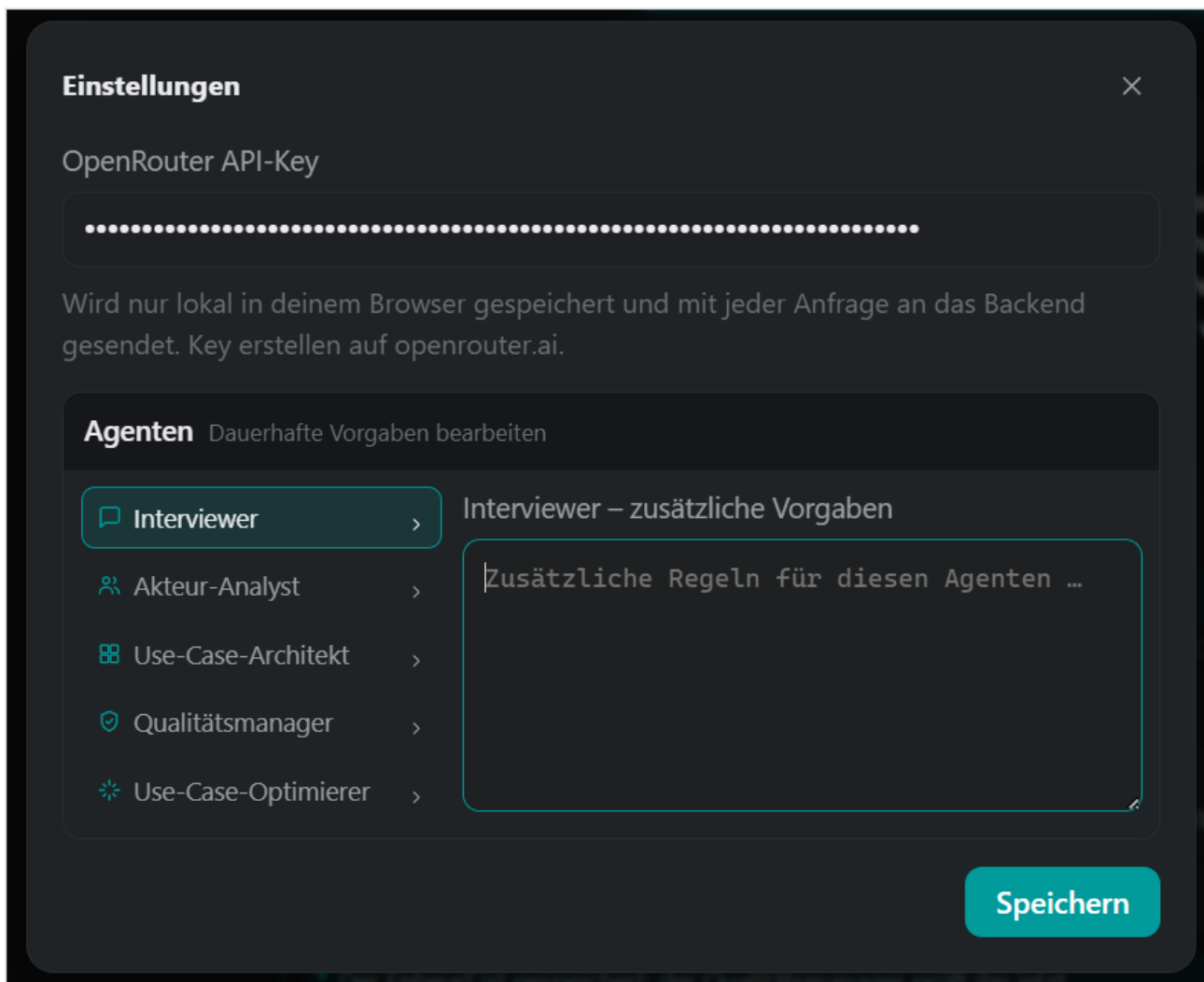


Abb. 4 — Einstellungen: OpenRouter-Key und dauerhafte Vorgaben je Agent.

4 · Prompt-Bar: Modell, Denkstufe, Sprache & Ausgabeordner

Über den Picker in der Prompt-Bar wählt man Modell und Denkstufe. Ein Hover-Flyout zeigt die vier Stufen **Niedrig** · **Mittel** · **Hoch** · **Max**. Links sitzt der Ausgabeordner-Chip, rechts das Mikrofon für die Spracheingabe.

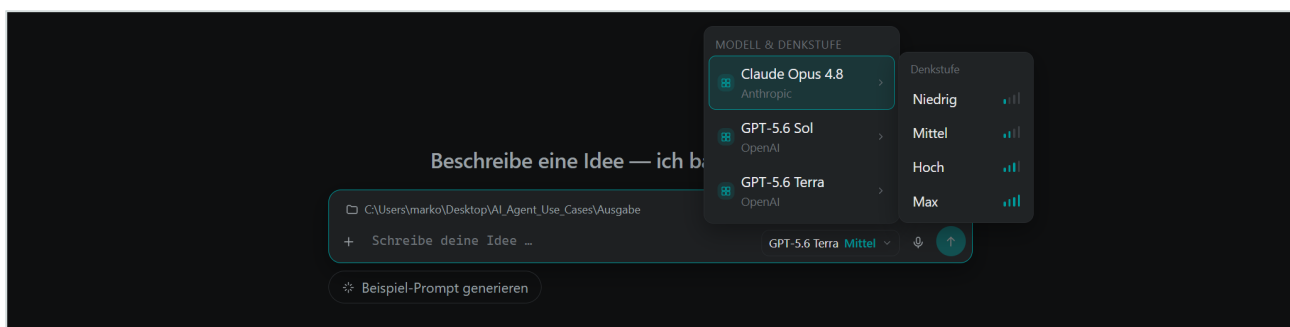


Abb. 5 — Modell- und Denkstufen-Auswahl, Spracheingabe und Ausgabeordner.

5 · Interviewer: gezielte Rückfragen

Der **Interviewer** versteht die Idee und stellt gezielte Rückfragen (hier auf Wunsch „maximal eine Rückfrage“). Sein Gedankenstrom (GEDANKEN) ist einsehbar, die Rückfragen erscheinen mit den bereits gegebenen Antworten, gefolgt von einer Zusammenfassung.

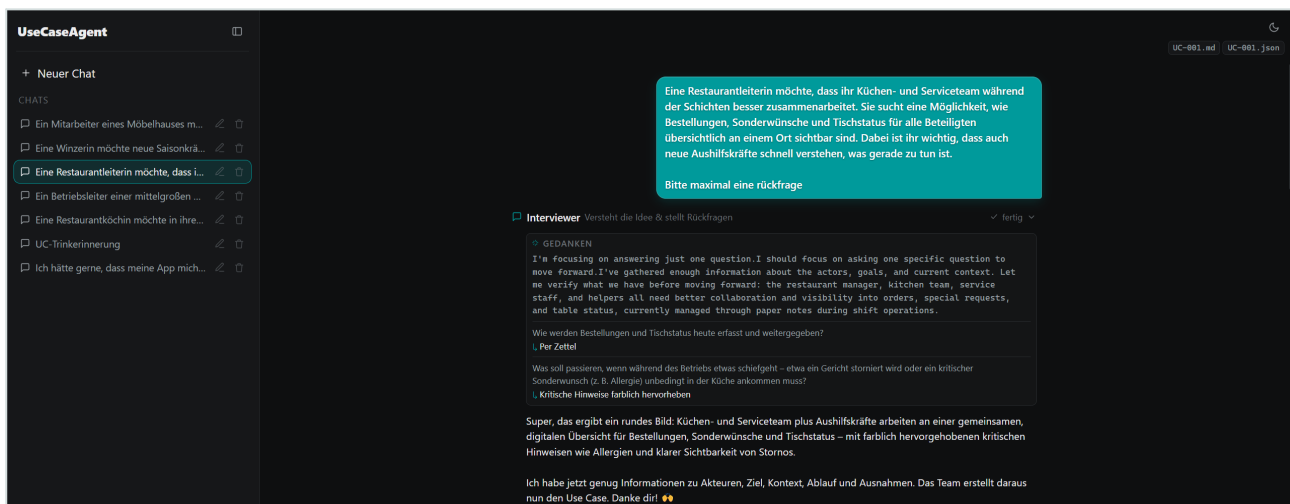


Abb. 6 — Interviewer mit sichtbarem Reasoning und Rückfragen.

6 · Use-Case-Architekt: Entwurf entsteht

Der **Use-Case-Architekt** baut aus den gesammelten Informationen einen ersten Entwurf. Die erzeugte Datei erscheint im Claude-Code-Stil als „Erstellt UC-001_draft.tmp.json +105“ mit Zeilenzähler und aufklappbarer Vorschau des Inhalts (Ziel, primärer Akteur, Stakeholder, Vorbedingungen ...).

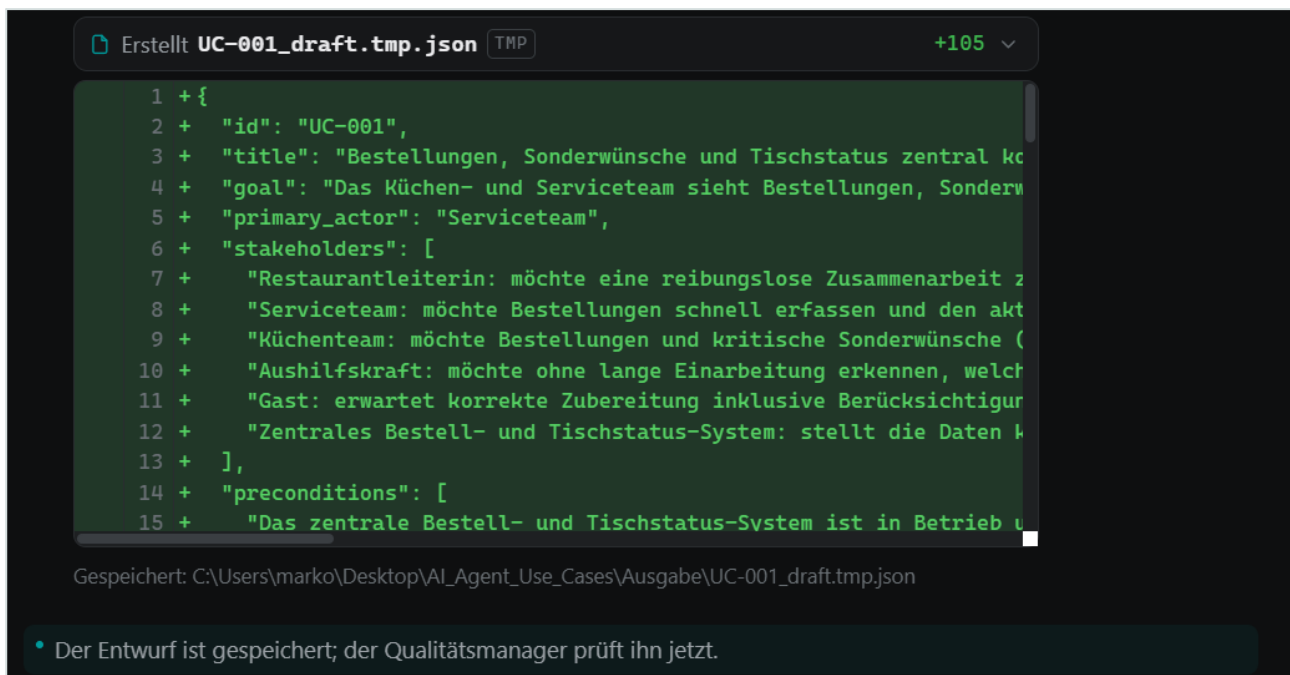


Abb. 7 — Use-Case-Architekt erstellt den TMP-Entwurf.

7 · Der vollständige Agenten-Ablauf im Chat

Alle Schritte in einer Ansicht: die **Übergaben** (z. B. Interviewer → Akteur-Analyst, Use-Case-Architekt → Qualitätsmanager) werden als eigene, farblich abgesetzte Ereignisse dargestellt. Jeder Agent zeigt Titel, Kurzbeschreibung, Status (fertig) und seinen Gedankenstrom.

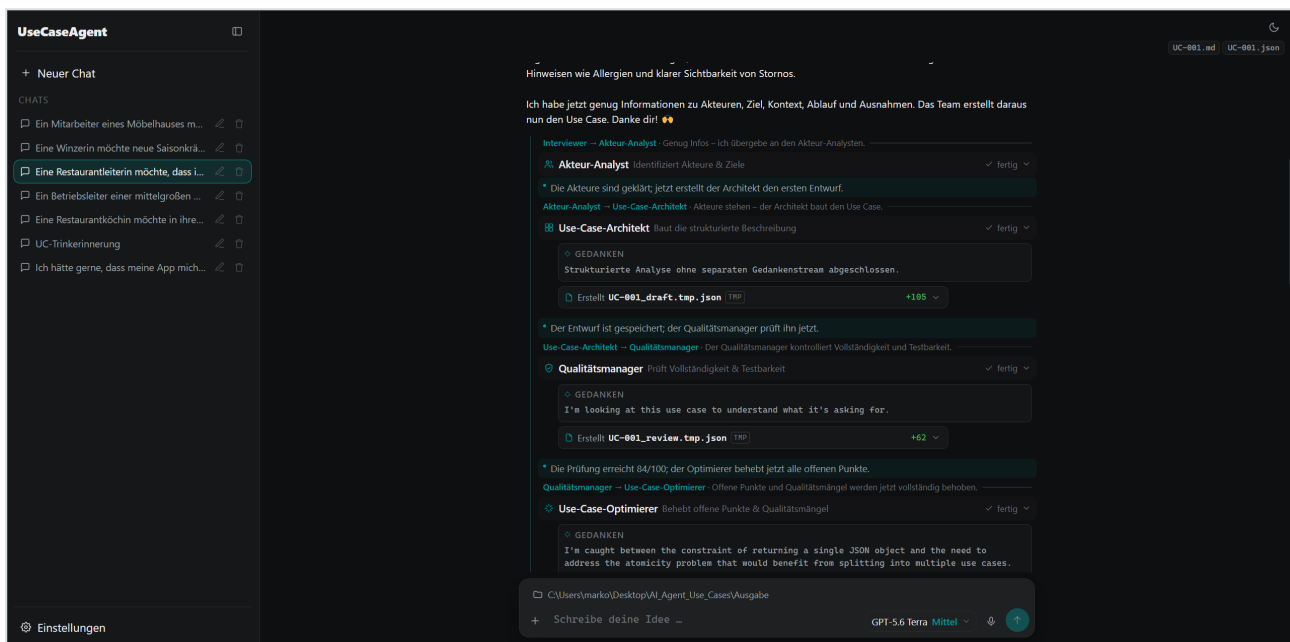


Abb. 8 — Vollständiger Agenten-Ablauf mit Übergaben.

8 · Qualitätsprüfung & Optimierung (Diff-Ansicht)

Der **Qualitätsmanager** bewertet Vollständigkeit und Testbarkeit und vergibt einen Score (hier 84/100), inklusive strukturierter *issues* (z. B. Kategorie „Atomarität“, Schweregrad „hoch“). Der **Use-Case-Optimierer** behebt die Punkte und bearbeitet den Entwurf — sichtbar als grün/rote Diff-Ansicht (+69 -40).

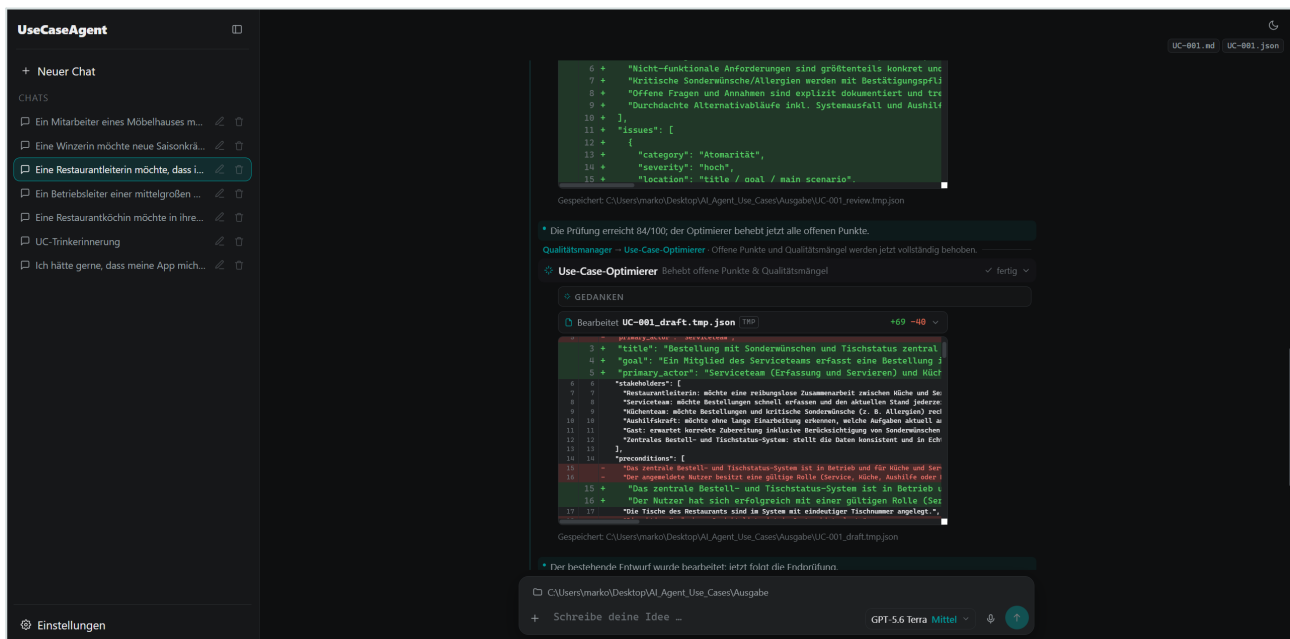


Abb. 9 — Qualitätsbewertung und Optimierer mit Diff-Ansicht.

9 · Endprüfung, Export & Aufräumen

Nach der Endprüfung wird die geprüfte finale Fassung gespeichert. Die temporären Entwürfe werden **automatisch gelöscht** („Gelöscht UC-001_draft.tmp.json -134“), und es bleiben die sauberen Enddateien **UC-001.md** und **UC-001.json**.

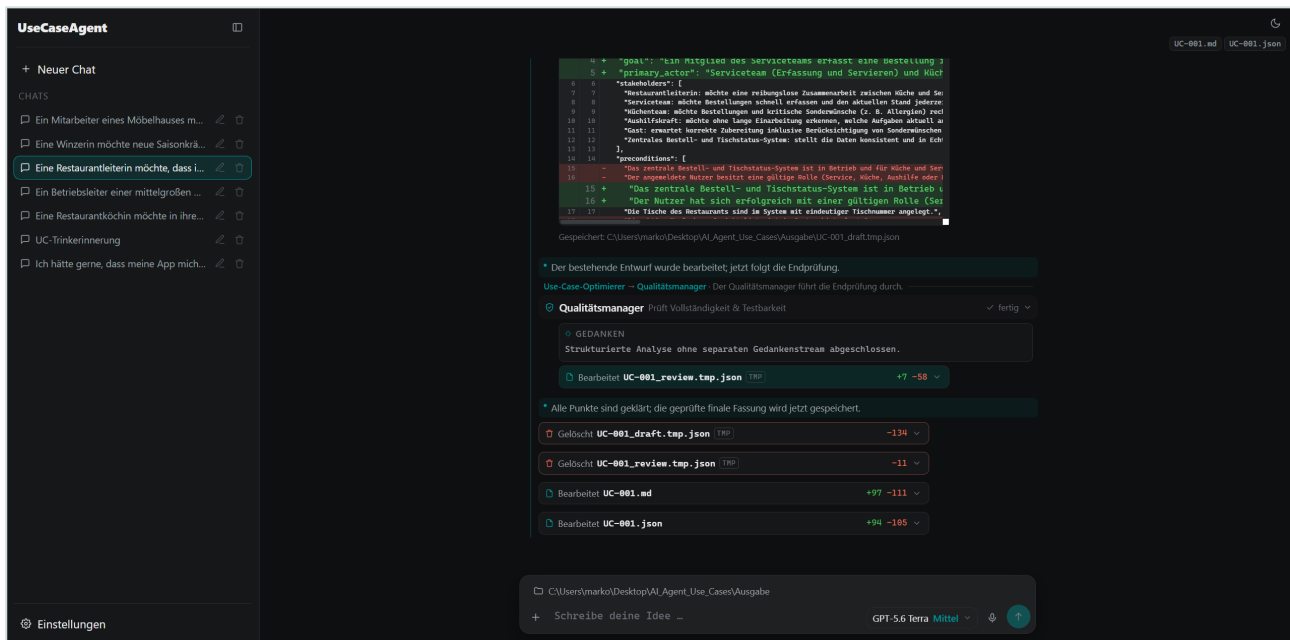


Abb. 10 — Endprüfung, Export und automatisches Aufräumen der TMP-Dateien.

10 · Ergebnis: der fertige Use Case

Am Ende meldet der Chat kompakt: „Der Use Case ... wurde erstellt und geprüft.“ mit anklickbarer Ergebnisdatei und der reinen Bearbeitungszeit (hier 5 Min. 1 Sek.).

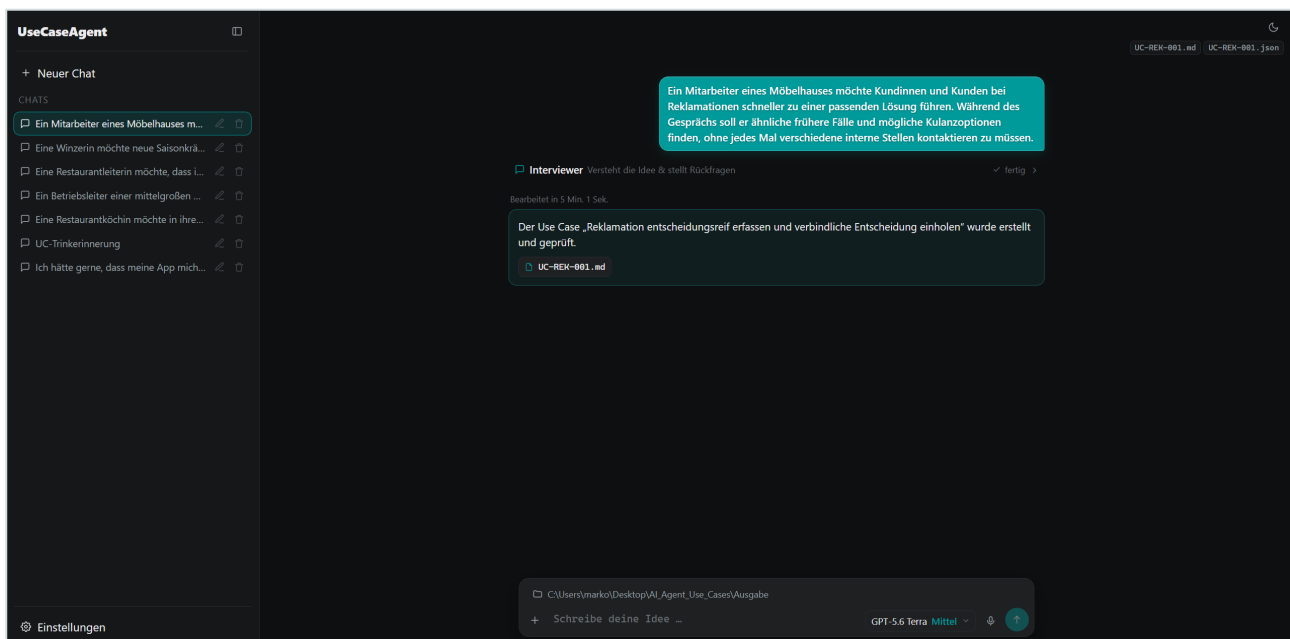


Abb. 11 — Fertig gestellter Use Case im Chat.

11 · Vorschau-Panel

Der fertige Use Case lässt sich direkt neben dem Chat als gerendertes Markdown ansehen — mit Titel, Ziel, primärem Akteur, Auslöser sowie „Stakeholder & Interessen“.

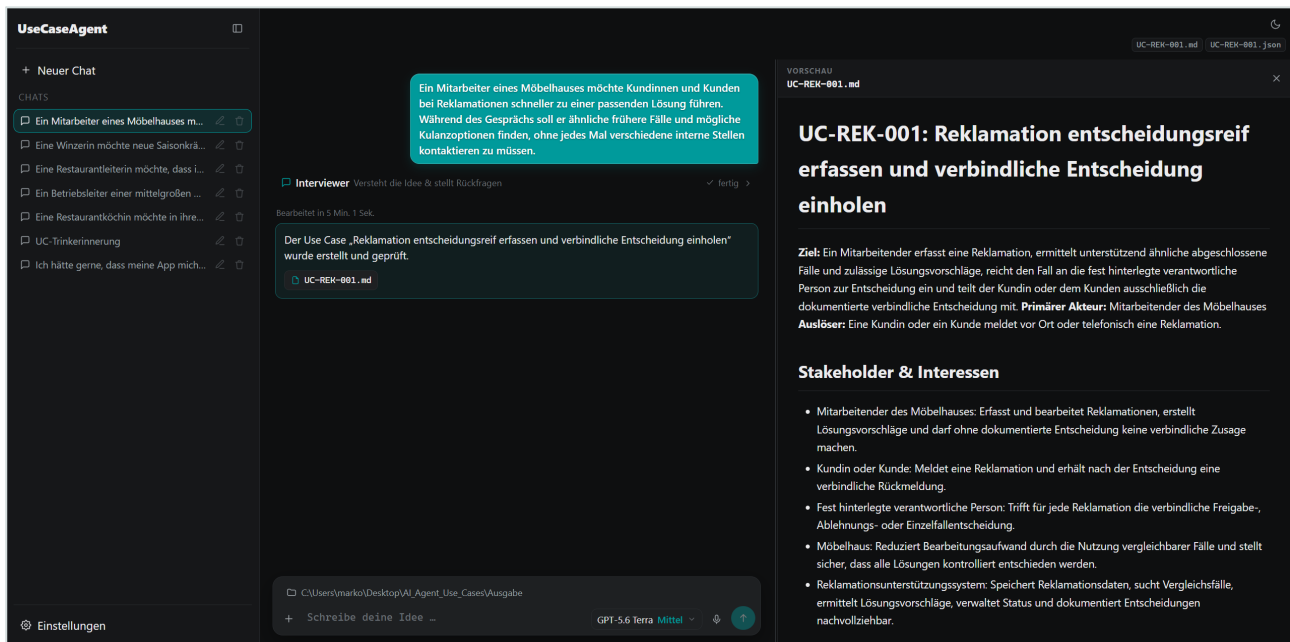


Abb. 12 — Use-Case-Vorschau im dunklen Theme.

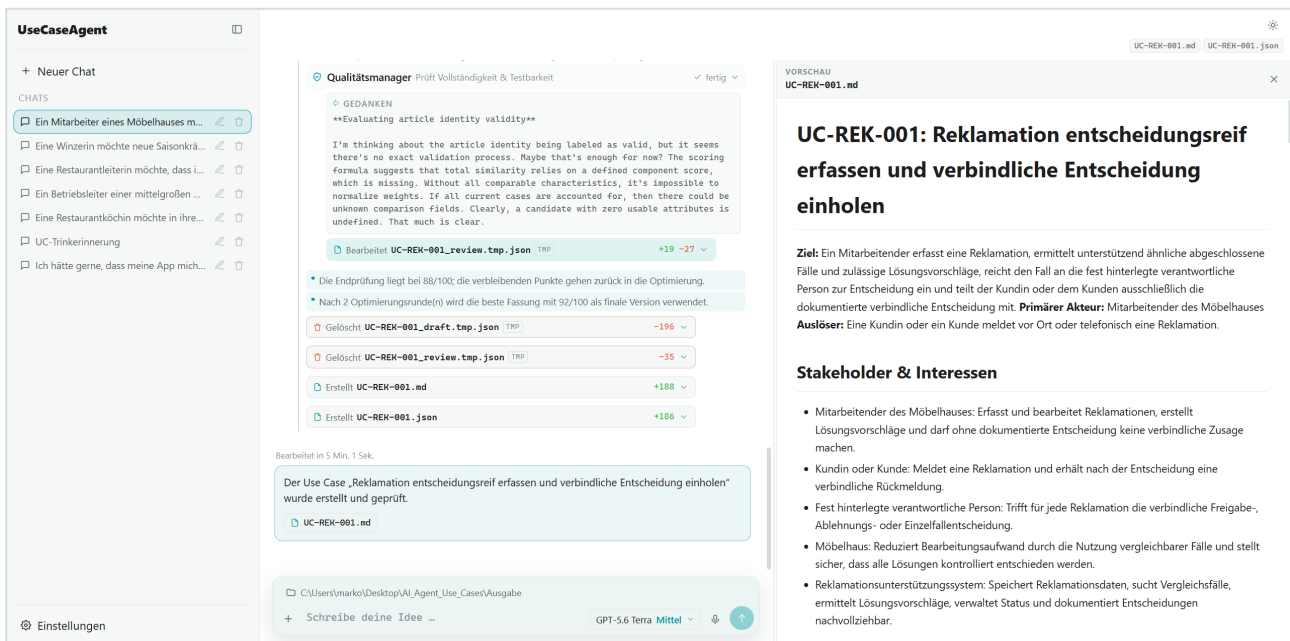


Abb. 13 — Use-Case-Vorschau im hellen Theme.

Beispiel: ein vollständiger Durchlauf

Als durchgehendes Beispiel dient die laienhafte Idee eines Möbelhaus-Mitarbeiters:

„Ein Mitarbeiter eines Möbelhauses möchte Kundinnen und Kunden bei Reklamationen schneller zu einer passenden Lösung führen. Während des Gesprächs soll er ähnliche frühere Fälle und mögliche Kulanzoptionen finden, ohne jedes Mal verschiedene interne Stellen kontaktieren zu müssen.“

Der Ablauf im Hintergrund:

1. **Interviewer** klärt offene Punkte mit gezielten Rückfragen.
2. **Akteur-Analyst** leitet Akteure und Ziele ab.
3. **Use-Case-Architekt** erstellt den Entwurf UC-REK-001_draft.tmp.json.
4. **Qualitätsmanager** prüft und bewertet (Endprüfung 88/100).
5. **Use-Case-Optimierer** verbessert die offenen Punkte — nach 2 Optimierungsrunden wird die beste Fassung mit **92/100** als finale Version übernommen.
6. **Export**: UC-REK-001.md + UC-REK-001.json; die TMP-Entwürfe werden gelöscht.

Ergebnis-Titel: „**UC-REK-001: Reklamation entscheidungsreif erfassen und verbindliche Entscheidung einholen**“ — reine Bearbeitungszeit 5 Min. 1 Sek.

Technische Entscheidungen

- **Streaming (SSE)**: das Backend streamt den gesamten Agenten-Ablauf als Ereignisse; das Frontend rendert Denken, Übergaben und Ergebnisse live.
- **Modularität**: jeder Agent kapselt eine Teilaufgabe mit eigenem Prompt und eigener Verantwortung und ist einzeln austausch- und erweiterbar.
- **Zwischenstände als TMP-Dateien**: Entwürfe und Reviews werden transparent als .tmp.json abgelegt und am Ende automatisch entfernt.
- **Kein Frontend-Build**: bewusst reines HTML/CSS/JS — läuft ohne Node/Toolchain.
- **Provider-Layer**: der OpenRouter-Zugriff ist in einer eigenen Client-Schicht isoliert; Modelle und Denkstufen sind zentral konfigurierbar.

Installation & Start

```
# 1. (optional) virtuelle Umgebung
python -m venv .venv
.venv\Scripts\activate           # Windows • source .venv/bin/activate (macOS/Linux)

# 2. Abhängigkeiten
pip install -r requirements.txt

# 3. Starten
py run.py                       # oder: py -m uvicorn backend.main:app --port 8000
```

Anschließend **http://localhost:8000** öffnen und unten links unter **Einstellungen** einen OpenRouter-Key (sk-or-...) hinterlegen. Eine vollständige Bildschirmaufnahme des Ablaufs liegt als 2026-07-22 15-17-35.mp4 im Projektordner bei.